DESAFIO 03

A Caixa de Lembranças

A vó anda esquecendo as histórias que sempre contou.
O sistema que lembra junto com ela é com você.

AMAZON BEDROCK · AMAZON S3 · IAM ROLE · RAG · AMAZON EC2

01 Contexto

A **Dona Cecília** tem 82 anos, faz o melhor bolo de laranja do bairro e conta as melhores histórias da família: a enchente de 1967, o casamento no salão paroquial, o gato Frajola que chegou numa noite de geada e ficou quinze anos. Sempre contou. Só que ultimamente as histórias começaram a escapar: ela troca os anos, esquece os nomes, para no meio de um causo com os olhos procurando o resto.

No último feriado, a família inteira se reuniu na casa dela e fez um mutirão bonito: um fim de semana ouvindo e digitando. Cada história, cada receita, cada carta virou um arquivinho de texto. Dez lembranças, salvas numa pasta chamada `lembrancas/`.

Aí a família olhou para a pasta e travou. Foi quando a sua prima, a que organizou o mutirão, mandou mensagem no grupo:

Grupo: Família Cecília 💐

Gente, conseguimos!!! Tá tudo digitado, as 10 lembranças da vó, revisadas.

Agora falta a parte que eu não sei fazer. Eu queria um negócio tipo um chat, sabe? Que a gente pergunta "vó, quantos ovos vão no bolo?" e ele responde **do jeitinho dela**, usando o que a gente digitou. E que MOSTRE de qual lembrança tirou a resposta, pra gente confiar.

Ah, e uma coisa importante, que eu vi num vídeo: essas IA inventam coisa. **Na nossa não pode.** Se não estiver nas lembranças, é pra dizer "isso eu não lembro, meu bem" e pronto. Inventar memória pra vó é a única coisa proibida aqui.

@você tu não faz aquele curso de nuvem? 👀

Você faz. E sabe exatamente o que ela descreveu sem saber o nome: um sistema de **RAG**: busca nas lembranças + um modelo que escreve com a voz da vó, admitindo quando não sabe.

O aniversário da Dona Cecília é daqui a duas semanas. A família quer mostrar a Caixa funcionando na festa.

02 O que temos?

```
desafio03-caixa-de-lembrancas/
├── lembrancas/       → as 10 lembranças da vó, em .md (o "acervo")
├── backend/          → a API do RAG (a IMAGEM já vem pronta no GHCR)
└── frontend/dist/    → o chat, JÁ BUILDADO (config do endereço em runtime)
```

O repositório: https://github.com/Liga-de-Computacao-em-Nuvem-PUCRS/desafio03-caixa-de-lembrancas

O backend já sabe fazer a parte de IA, e faz na frente de quem quiser ler, sem biblioteca mágica: fatia as lembranças, gera **embeddings** com o Titan (Amazon Bedrock), busca por **similaridade de cosseno** e responde com um modelo **Claude**, tudo comentado como aula em `src/rag.js` e `src/indice.js`. A honestidade que a sua prima exigiu está no prompt e num limiar de similaridade: sem lembrança relevante, a vó diz que não lembra.

O que **não** veio: a **policy da IAM Role** (nem a role), os modelos habilitados, o diagrama, a estimativa de custos e qualquer configuração de nuvem. E uma ausência proposital: **não há nenhum par de chaves de acesso neste desafio**. Repare nisso lendo o `.env.example`.

03 Sua missão

Colocar a Caixa de Lembranças no ar: as lembranças num bucket S3, a API numa EC2 que se autentica por **IAM Role**, sem nenhuma credencial em arquivo, e o Bedrock respondendo com a voz da Dona Cecília.

01 Subir as lembranças para um bucket S3

Bucket novo, **privado** (as memórias da vó não são página pública), com os 10 arquivos de `lembrancas/` dentro. Critério: os objetos aparecem no console, e o nome do bucket está anotado para as variáveis da API.

AMAZON S3

02 Habilitar os modelos no Bedrock

Em **Bedrock → Model access**, na região **us-east-1**: o **Titan Text Embeddings V2** (Amazon) e o **Claude Haiku** (Anthropic). Habilitar não custa nada; a cobrança é por uso. Critério: os dois aparecem como *Access granted*, e você sabe dizer qual dos dois faz o quê no sistema.

BEDROCK · MODEL ACCESS

03 Escrever a policy e criar a IAM Role da instância

Este é o coração do desafio, e o que não veio pronto. O README lista exatamente o que o backend chama na AWS; a sua policy cobre **isso e nada mais**: menor privilégio de verdade. Depois, a role vira um *instance profile* anexado à EC2. Critérios: o `/health` mostra `bucket: "ok"` sem nenhuma variável de credencial em lugar nenhum, e você sabe explicar por que `s3:ListBucket` e `s3:GetObject` apontam para **ARNs diferentes**.

IAM ROLE · LEAST PRIVILEGE

04 Subir a EC2 com a role e o container

Instância no Free Tier, com a role anexada, Docker, e o container puxado do GHCR com as variáveis do desafio. Security Group: SSH do seu IP, 3000 aberta para o site. Critério: `curl IP:3000/health` responde de fora.

AMAZON EC2 · IMDS

05 Indexar e sabatinar a vó

Chame `POST /api/reindexar` e valide com perguntas de resposta objetiva: *quantos ovos vão no bolo de laranja?* (três), *em que ano foi a enchente?* (1967), *quantas cartas estão na lata?* (47). E o teste que a sua prima faria: pergunte algo que **não está** nas lembranças (time do coração, por exemplo) e confira que a vó diz que não lembra, em vez de inventar. Critério: as três certas **e** a recusa honesta, com os trechos mostrados na tela fazendo sentido.

RAG · EMBEDDINGS

06 Publicar o chat no S3

Bucket de site estático (esse sim, público), `dist/config.js` editado com o IP da EC2 e a porta. Critério: qualquer pessoa da família abre o endereço e conversa com a Caixa.

AMAZON S3 · STATIC HOSTING

07 Entregar o diagrama e a estimativa de custos para a família

O diagrama mostra o caminho de uma pergunta (navegador, EC2, Bedrock, S3) e onde a IAM Role entra. A estimativa explica **a conta de tokens para leigo**: quanto custa indexar tudo, quanto custa uma pergunta, quanto custa a festa inteira com a família perguntando sem parar. (Spoiler que você vai demonstrar: menos que um pão de queijo.)

> ***As lembranças da vó não são dataset público.** Bucket privado, sem "Block public access" desligado, e nada de URL aberta. A única coisa pública neste desafio é o site.

ARQUITETURA · TOKENS

04 O combinado

- Região: **us-east-1**, do começo ao fim, inclusive o Model access.
- **AWS Budget de US$ 5** configurado antes de qualquer recurso.
- Ao terminar: instância encerrada, role e instance profile apagados, buckets esvaziados e apagados. Model access pode ficar: parado, não custa.

05 Desafio ponto de arremate

A festa foi um sucesso. A tia Marlene monopolizou a Caixa por quarenta minutos. Mas na segunda-feira a sua prima mandou mensagem de novo:

> A EC2 reiniciou de madrugada (eu vi no email da AWS) e a Caixa levou uns dois minutos "reaprendendo" tudo quando voltou. A vó não pode reaprender as próprias lembranças toda vez que acorda, né? E... isso custa de novo cada vez, não custa?

Custa. A cada boot, a API refaz **todos** os embeddings: paga de novo pelo que já tinha calculado. Costureira que é, a Dona Cecília explicaria melhor que ninguém: costura sem **arremate** se desmancha no primeiro puxão. Memória sem persistência também.

08

Fazer a memória de longo prazo sobreviver ao reinício

O backend já sabe fazer a parte dele: com `SALVAR_INDICE=true`, ele grava o índice de vetores no próprio bucket e o recarrega no boot. Ative a flag e faça funcionar. O detalhe: isso exige **uma permissão a mais** na sua policy. Descubra qual pelo erro, adicione **só ela**, e documente a mudança. Estender uma policy conscientemente é o exercício; trocar por `s3:*` é reprovação moral.

AMAZON S3 · IAM POLICY

Você dá o ponto de arremate quando

- Um reinício da instância volta a responder **sem** refazer embeddings, e você aponta no log a linha que prova ("índice restaurado do bucket").
- O diff da sua policy tem **exatamente uma ação** a mais, no recurso certo.
- Você sabe dizer quanto a família economiza por boot, em chamadas e em centavos, com o índice persistido.

> **Dica da vó:** "credencial é que nem chave de casa, meu bem: quanto menos cópia espalhada, melhor". Procure por **instance profile**: uma instância pode ter identidade própria, com credenciais temporárias que a AWS rotaciona sozinha, sem nenhuma chave sua envolvida. E se o SDK dentro do container reclamar que não achou credencial nenhuma, lembre do desafio anterior: conte os **saltos** até o serviço de metadados.